CARE Documentation

2026-06-21

Contents

CAREDocumentation **2**
- Installing dependencies . . . 2
- Building Docs . . . 2

Passwords **2**

Important Locations **2**
- Websites . . . 2
- Git Repositories (on disk) . . . 3
- Initial Setup . . . 3
- Git Repositories (on internet) . . . 3

Reading SystemD Logs for CARE Services **4**
- Services . . . 4
- Checking Service Status . . . 4
- Reading Logs . . . 4
- Justfiles . . . 5
- Restarting a Service . . . 5

CARERabiesStats **5**
- What It Does . . . 5
- Environment Variables . . . 5
- Managing the Service . . . 6
- Running Reports . . . 6
- Owner Data Notes . . . 6
- Data Files . . . 7
- Build . . . 7

CARECatStatus **7**
- What It Does . . . 7
- Managing the Service . . . 7
- User Management . . . 8
- Login . . . 8
- Deploying Updates . . . 8
- Configuration . . . 8
- API & Docs . . . 8

Meowderall **8**
- What It Does . . . 9
- Tech Stack . . . 9
- Managing the Service . . . 9

Deploying Updates . . . 9
Build . . . 9

CAREShelterDonationDataAggregation **10**
What It Does . . . 10
Using the Web App . . . 10
Output Fields . . . 10
Managing the Service . . . 10
Deploying Updates . . . 11
GCP Deployment . . . 11

CARE Donation Thermometer **11**
What It Does . . . 11
Using the Admin Portal . . . 12
Admin Key Management . . . 12
Embedding in Emails . . . 12
Managing the Service . . . 12
Deploying Updates . . . 13
Environment Variables . . . 13
Firestore Management . . . 13
API . . . 13

CAREDocumentation

This repository contains documentation for CARE staff to use if they need to modify or update CARE Rust services.

Installing dependencies

To install dependencies, run the following command:

```
just install-deps-cachyos
```

Building Docs

Docs are built with pandoc, by stitching together multiple *.md files.

```
just compile-docs
```

Passwords

Passwords are stored in a password manager by Henry Post.

Please contact him if you need access to any system.

Note: Some services have `.env` files in the Git repository that contain passwords.

Important Locations

Websites

Internet

- https://test1.drakonix.systems
- https://test2.drakonix.systems

- https://test3.drakonix.systems
- https://test4.drakonix.systems
- https://carerabiesstats.drakonix.systems/
- https://carecatstatus.drakonix.systems/
- https://meowderall.drakonix.systems/
- https://carethermometer.drakonix.systems/
- https://donationaggregator.drakonix.systems/

Local

These should be accessed if physically using the CARE server laptop.

These local connections are routed here https://github.com/meltingscales/drakonix.systems/blob/master/nginx/drakonix.systems.conf#L54 as well, under `proxy_pass`.

- http://localhost:3003 - CAREShelterDonationDataAggregation
- http://localhost:3002 - animal-shelter-donation-thermometer
- http://localhost:3001 - Meowderall
- http://localhost:3005 - CARERabiesStats
- http://localhost:3007 - CARECatStatus

Git Repositories (on disk)

Please note that `~` means "user's home directory".

By default this is `/home/careops/`.

All these repositories live under `/home/careops/Git/*`.

- ~/Git/CARERabiesStats
- ~/Git/CARECatStatus
- ~/Git/Meowderall
- ~/Git/CAREShelterDonationDataAggregation
- ~/Git/animal-shelter-donation-thermometer

Initial Setup

Clone all CARE repositories onto the server:

```
cd ~/Git

git clone git@github.com:meltingscales/CARERabiesStats.git
git clone git@github.com:meltingscales/CARECatStatus.git
git clone git@github.com:meltingscales/meowderall.git
git clone git@github.com:meltingscales/CAREShelterDonationDataAggregation.git
git clone git@github.com:meltingscales/animal-shelter-donation-thermometer.git
```

Git Repositories (on internet)

- https://github.com/meltingscales/CARERabiesStats
- https://github.com/meltingscales/CARECatStatus
- https://github.com/meltingscales/meowderall
- https://github.com/meltingscales/animal-shelter-donation-thermometer
- https://github.com/meltingscales/CAREShelterDonationDataAggregation

Reading SystemD Logs for CARE Services

Each CARE service runs as a SystemD unit on the server.

All repositories live under `/home/careops/Git/` on the CARE server laptop.

Services

Service Name	Repository Directory
`carerabiesstats`	`~/Git/CARERabiesStats`
`carecatstatus`	`~/Git/CARECatStatus`
`meowderall`	`~/Git/Meowderall`
`donationaggregator`	`~/Git/CAREShelterDonationDataAggregation`
`carethermometer`	`~/Git/animal-shelter-donation-thermometer`

Checking Service Status

`systemctl status SERVICE`

Example:

`systemctl status carerabiesstats`

This shows whether the service is running, its last few log lines, and its PID.

Reading Logs

Basic log read

`journalctl -xu SERVICE`

The flags: - `-x` — add explanatory help text to log entries - `-u SERVICE` — filter to one specific unit/service

Example:

`journalctl -xu carerabiesstats`

Follow logs in real time

`journalctl -xu SERVICE -f`

Useful when actively debugging — new log lines print as they arrive.

Show logs since last boot

`journalctl -xu SERVICE -b`

Filter by time

```
# Logs from the last hour
journalctl -xu SERVICE --since "1 hour ago"

# Logs between two times
journalctl -xu SERVICE --since "2024-01-01 08:00" --until "2024-01-01 09:00"
```

Show only errors

`journalctl -xu SERVICE -p err`

Priority levels (from most to least severe): `emerg`, `alert`, `crit`, `err`, `warning`, `notice`, `info`, `debug`.

Show N most recent lines

```
journalctl -xu SERVICE -n 100
```

Justfiles

Each CARE service repository usually has a `justfile` with common maintenance commands.

```
# List available commands
just --list

# Or browse the file directly
less justfile
```

Common things Justfiles may include:

- Restarting the service
- Resetting passwords
- Running database migrations or maintenance
- Pulling the latest code and redeploying

Restarting a Service

If a service is broken or needs a restart after a code change:

```
sudo systemctl restart SERVICE
```

To stop or start:

```
sudo systemctl stop SERVICE
sudo systemctl start SERVICE
```

CARERabiesStats

Tools for formatting and submitting rabies vaccination data to Cook County's Animal Control & Rabies (ARC) program.

- **Live URL**: https://carerabiesstats.drakonix.systems/
- **Repo**: `~/Git/CARERabiesStats`
- **SystemD service**: `care-rabies-stats`
- **Port**: 3005
- **Contacts**: Karin, Martha
- **Cook County ARC phone**: 708-974-6140
- **Cook County ARC email**: samantha.may@cookcountyil.gov (test submissions)

What It Does

C.A.R.E. is enrolled in Cook County's Automatic Certificate Entry (ACE) program, which eliminates paper certificate submissions.

This tool pulls rabies vaccination records from ShelterLuv and formats them into Cook County's required tab-separated format (22 columns).

Environment Variables

Copy `.env.example` to `.env` and fill in values:

```
SHELTERLUV_API_KEY=replaceme1234
CARE_RABIES_STATS_AUTH_KEY=replace-with-a-uuid-here
```

- `SHELTERLUV_API_KEY` — get from ShelterLuv configuration
- `CARE_RABIES_STATS_AUTH_KEY` — UUID used to authenticate report downloads via web interface

Managing the Service

```
# Check service status
just systemd-status

# Follow live logs
just systemd-logs

# Restart service (requires sudo)
sudo just systemd-restart

# Install service from scratch (requires sudo)
sudo just systemd-install

# Remove service (requires sudo)
sudo just systemd-uninstall
```

Running Reports

API Mode (Recommended)

Pulls data directly from ShelterLuv. Requires `SHELTERLUV_API_KEY` in environment or `.env` file.

```
# Test run (50 records, logs to file)
just api-test

# Fetch all vaccinations, save to output.txt with log
just api-log output.txt run.log

# Fetch only 2025 vaccinations
just api-2025 output.txt run.log

# Fetch since a specific Unix timestamp
just api-since 1735711200 output.txt
```

CSV Mode

If you have a CSV export from ShelterLuv:

```
just csv input.csv output.txt
```

Owner Data Notes

The tool finds owner information in this priority order:

1. **Adopted animals** → uses adopter information
2. **Foster animals** → uses foster parent information
3. **Shelter animals** → owner fields left blank

Many animals vaccinated at C.A.R.E. are still in shelter custody, so many records will have **empty owner fields**. The tool prints a breakdown after processing.

> Verify with Cook County whether records with blank owner fields are acceptable for submission.

Data Files

Breed/color code mapping files (`data/breed list.xlsx`, `data/color list.xlsx`) are stored in `./data/` and are not in git (gitignored). These files must be present on the server.

Build

```
just build          # build everything
just build-cli      # CLI tool only
just build-web      # web server only
```

CARECatStatus

Real-time collaborative cat status board for shift staff at CARE Animal Shelter.

- **Live URL**: https://carecatstatus.drakonix.systems/
- **Repo**: `~/Git/CARECatStatus`
- **SystemD service**: `care-cat-status`
- **Port**: 3007

What It Does

Multiple staff members can view and update cat statuses simultaneously — changes sync across all connected devices instantly via WebSocket.

Per-cat fields:

Field	Values
`name`	Cat's name
`color`	`green` / `yellow` / `blue`
`location`	`adoption center` / `foster`
`notes`	Free text
`food_notes`	Free text

Also installable as a PWA on phones and tablets.

Managing the Service

```
# Check service status
just systemd-status

# Follow live logs
just systemd-logs

# Restart service (requires sudo)
sudo systemctl restart care-cat-status

# Install service from scratch (requires sudo)
sudo just systemd-install

# Remove service (requires sudo)
sudo just systemd-uninstall
```

User Management

Authentication is **disabled when no users exist**. Once any user is added, everyone must sign in with username + PIN.

```
# Add a user
just add-user firstname-lastname 1234

# Change a user's PIN
just modify-user firstname-lastname 5678

# Rename a user
just rename-user old-name new-name

# Delete a user
just delete-user firstname-lastname

# List all users
just list-users
```

Usernames must be lowercase letters and hyphens only (e.g. `jane-doe`).

Login

Open the app in a browser. If users exist, a sign-in screen appears:

- Type your `firstname-lastname` in the username field
- Enter your PIN on the numpad (click buttons or press digit keys / Backspace / Enter)

Deploying Updates

```
cd ~/Git/CARECatStatus
just build
sudo systemctl restart care-cat-status
```

Configuration

Environment Variable	Default	Description
`PORT`	`3000`	HTTP/WebSocket port
`DATABASE_URL`	`cats.db`	SQLite database path

API & Docs

- OpenAPI spec: `/openapi.json`
- Swagger UI: `/docs`

Meowderall

Digital cat medication schedule tracker for animal shelters.

- **Live URL**: https://meowderall.drakonix.systems/
- **Repo**: `~/Git/meowderall`
- **SystemD service**: `meowderall`
- **Port**: 3001

What It Does

SPA (Single-Page Application) web app — primarily used on iPads via touch screen — that helps shelter staff manage cat medication schedules.

Key behaviors:

- **No backend / no server-side storage** — all data lives in browser `localStorage`
- Admins enter a 4-digit PIN to modify AM/PM initial fields; all other fields are world-readable/writable
- Can export data as PDF or CSV

  Important: Because there is no backend, data is stored locally in the browser. Clearing browser data or switching devices will lose data. Staff should export regularly if persistence matters.

Tech Stack

- **Elm** (compiled to JavaScript)
- Static files served by nginx (via Docker)

Managing the Service

```
# Check service status
just systemd-status

# Follow live logs
just systemd-logs

# Restart service (requires sudo)
sudo systemctl restart meowderall

# Install service from scratch (requires sudo)
sudo just systemd-install

# Remove service (requires sudo)
sudo just systemd-uninstall
```

Deploying Updates

```
cd ~/Git/meowderall

# Build and deploy to GCP Cloud Run
just gcp-deploy-all

# Or rebuild the Elm app and restart the local service
just build-release
sudo systemctl restart meowderall
```

Build

```
# Development build
just build

# Optimized production build
just build-release

# Run local dev server (port 8081)
just dev
```

CAREShelterDonationDataAggregation

Web tool to aggregate donation data from multiple platforms into a single DonorSnap-ready CSV.

- **Live URL**: https://donationaggregator.drakonix.systems/
- **Repo**: `~/Git/CAREShelterDonationDataAggregation`
- **SystemD service**: `care-shelter-donation`
- **Port**: 3003

What It Does

C.A.R.E. receives donations through multiple platforms. Each exports data differently, making DonorSnap imports painful. This tool accepts one consolidated Excel file and outputs a standardized CSV.

Supported sources:

- DonorSnap
- Qgiv
- PayPal
- Square
- ShelterLuv
- Facebook PayPal

Privacy: No data is stored. Files are processed in memory and immediately deleted.

Using the Web App

1. Go to the URL above (or `http://localhost:3003` locally)
2. Upload an Excel file (`.xls` or `.xlsx`) with sheets named exactly:
   - `DonorSnap`
   - `Qgiv`
   - `ShelterLuv`
   - `Square`
   - `Facebook PayPal`
3. Download the combined CSV, ready for DonorSnap import

Additional pages:

- `/about` — how the tool works
- `/mappings` — field mappings from each source to output
- `/faq` — frequently asked questions

Output Fields

The output CSV contains:

- First, Last, Company
- EMail, Phone
- Address1, Address2, Address3, City, State/Province, Zip/Postal Code, Country
- Salutation
- Donation Date, Amount, Donation Type, Payment Method
- DonationNote

Managing the Service

```
# Check service status
just systemd-status
```

```
# Follow live logs
just systemd-logs

# Restart service (requires sudo)
sudo systemctl restart care-shelter-donation

# Install service from scratch (requires sudo)
sudo just systemd-install

# Remove service (requires sudo)
sudo just systemd-uninstall
```

Deploying Updates

```
cd ~/Git/CAREShelterDonationDataAggregation

# GCP Cloud Run deployment (production)
just gcp-deploy-all

# Or rebuild locally
just build
sudo systemctl restart care-shelter-donation
```

GCP Deployment

The production service runs on GCP Cloud Run. To deploy:

```
export GCP_PROJECT="careshelterdonationdataagg"
export GCP_REGION="us-central1"

# Build Docker image, push to GCR, deploy to Cloud Run
just gcp-deploy-all

# View live logs from Cloud Run
just gcp-logs

# Get production URL
just gcp-url
```

CARE Donation Thermometer

Visual donation progress thermometer for CARE fundraising campaigns.

- **Live URL**: https://carethermometer.drakonix.systems/
- **Repo**: `~/Git/animal-shelter-donation-thermometer`
- **SystemD service**: `animal-shelter-thermometer`
- **Port**: 3002

What It Does

Tracks team donation totals and displays them as a visual thermometer image. The thermometer PNG can be embedded directly in fundraising emails.

Key features:

- Embeddable thermometer image at `/thermometer.png`

- Multiple teams with individual totals and leaderboard
- Web-based Admin Portal at `/admin` for configuration
- Data persists via Google Cloud Firestore (or in-memory for local dev)

Using the Admin Portal

Visit `/admin` and authenticate with the `THERMOMETER_EDIT_KEY`.

From the admin portal you can:

- Set organization name, campaign title, and fundraising goal
- Upload a CSV file with team donation data

CSV Format

```
name,image_url,total_raised
Team Alpha,https://example.com/alpha.jpg,1250.50
Team Beta,,2340.00
```

- `name` — team name (required)
- `image_url` — team logo URL (optional)
- `total_raised` — dollars raised (required)

Admin Key Management

The `THERMOMETER_EDIT_KEY` is a UUID required for all admin API calls.

```
# Generate a new key
just generate-key

# Set key in Cloud Run
just set-cloud-key <key>

# Rotate key in Cloud Run (prompts for confirmation)
just rotate-cloud-key

# Show current Cloud Run environment variables
just show-cloud-env
```

If no key is set, the server auto-generates one and prints it in the logs on startup:

```
WARN: THERMOMETER_EDIT_KEY not set, generated new key: <key>
```

Embedding in Emails

```
<img src="https://carethermometer.drakonix.systems/thermometer.png" alt="Donation Progress">
```

The image uses cache-busting headers so emails always show the latest totals.

Managing the Service

```
# Check service status
just systemd-status

# Follow live logs
just systemd-logs

# Restart service (requires sudo)
```

```
sudo systemctl restart animal-shelter-thermometer

# Install service from scratch (requires sudo)
sudo just systemd-install

# Remove service (requires sudo)
sudo just systemd-uninstall
```

Deploying Updates

```
cd ~/Git/animal-shelter-donation-thermometer

# Full GCP Cloud Run deployment
just gcp-deploy-all

# Or for local service
just build
sudo systemctl restart animal-shelter-thermometer
```

Environment Variables

Variable	Required	Description
`GCP_PROJECT`	No	GCP project ID — enables Firestore persistence
`THERMOMETER_EDIT_KEY`	No	UUID for admin auth (auto-generated if unset)
`PORT`	No	Server port (default: 8080)
`BASE_URL`	No	Base URL (default: `http://localhost:8080`)

Without `GCP_PROJECT`, data is stored in memory only and lost on restart.

Firestore Management

```
# Setup Firestore (one-time)
just firestore-setup

# Check Firestore status
just firestore-status

# Clear all Firestore data (DANGEROUS - prompts for confirmation)
just firestore-clear
```

API

- Swagger UI: `/openapi`
- Health check: `/health`
- Config (JSON): `/config`